# 14 Client-Server

In this chapter, we present and explain the Client-Server architectural style and how to specify it in SysADL. We specify the style using the structural and behavioral viewpoints. Finally, we illustrate the Client-Server style and how to use it with our running example.

You will learn:

- what is a Client-Server architectural style;
- what are its architectural elements, structure, and behavior.

## 14.1 Conceptual Overview

In a Client-Server style, components and connectors have a particular behavior. Components, called "clients", send requests to a component, called "server", and wait for a reply. A server component receives a request from a client and sends it the reply. We can use the Client-Server style to model a part of a system that has many components sending requests (clients) to another component (server) that offer services.

An example of client-server style is the use of a component – the client – that requests to the controller the average temperature. The controller is a server that provides the temperature value as a reply to the client component.

***Figure 1*** depicts the essential elements in a Client-Server configuration style. The *ClientCP* component sends requests to a server using the *ClientServerCN* connector. The *ServerCP* component offers services to reply the clients' requests. There is only one server and zero or more clients.


**Figure 1.** ***Client-Server*** **Overview**

Both components have a composite port. The *ClientPT* port is composed by one *out* port, *cq:QueryOPT*, to send a request to the server, and one *in* port *ca:AnswerIPT*, to receive the answer from the server. The *ServetPT* port is composed by one *in* port, *cq:QueryIPT*, to receive the request from the clients, and one *out* port *ca:AnswerOPT* to prove the answer.

*ClientServerCN* is a composite connector. It is composed of two components to transmit, the request and response. The *ClientServerQueryCN* connector links the *cq:QueryOPT* port to the *cq:QueryIPT* port. The *ClientServerAnswerCN* connect-

or links the *cq:AnswerOPT* port to the *cq:AnswerIPT* port. ***Figure 2*** shows the details of the composite ports and the two connectors of *ClientServerCN*. This configuration is an alternative visualization of the configuration illustrated in ***Figure 1***.

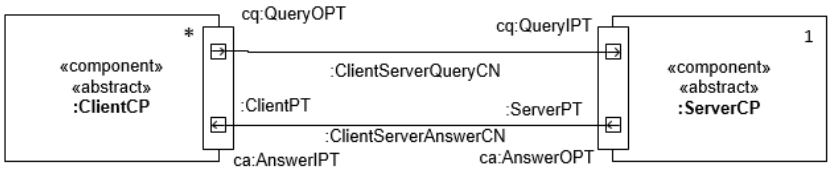


**Figure 2. The composite ports and connector in the *Client-Server* configuration.**

## 14.2 An Example of Client-Server in RTC System

To illustrate the use of a Client-Server in the *RTC* System, we add a new component that is a client to display the current room average temperature, shown in ***Figure 3***. *TemperatureDisplayCP* sends a query via the *ClientServerCN* composite connector and waits for the answer. The *ServerPT* port is a composite proxy port to an internal component that stores the last current average temperature.

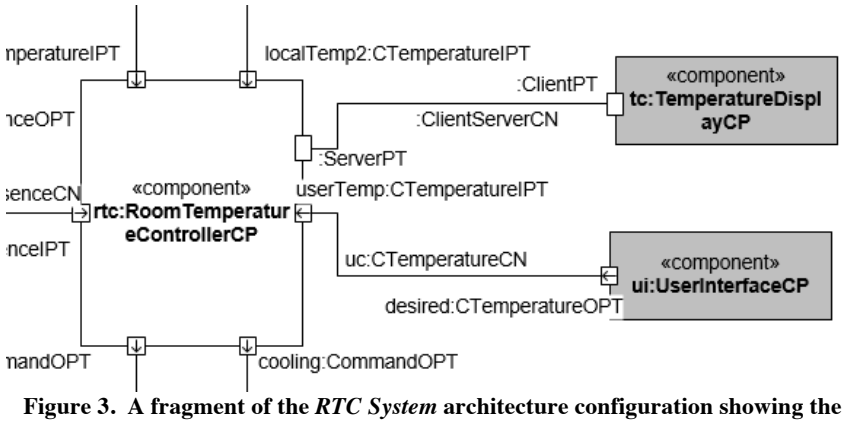


**Figure 3. A fragment of the *RTC System* architecture configuration showing the new TemperatureDisplayCP component.**

The *TemperatureStorageCP* internal component is the server in this example. It receives the average temperature from the *SensorsMonitorCP* component (not shown here) in its average port (see ***Figure 4***). When it receives a query in its *sq:QueryIPT* port, it sends the average temperature in its *sa:QueryOPT* port.

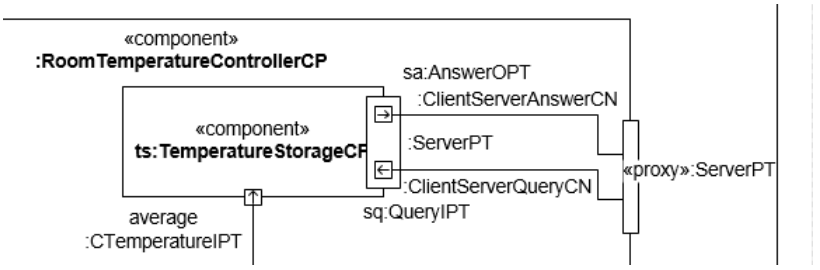


**Figure 4. The *TemperatureSotrageCP* component that acts as a server providing the current temperature.**

## 14.3 Client-Server Structural Viewpoint

We define the Client-Server architectural style using a *bdd*, as illustrated in ***Figure 5***. The Client-Server Architecture Style (*ClientServerARCH*) is composed of one *ServerCP* component and zero or more *ClientCP*.

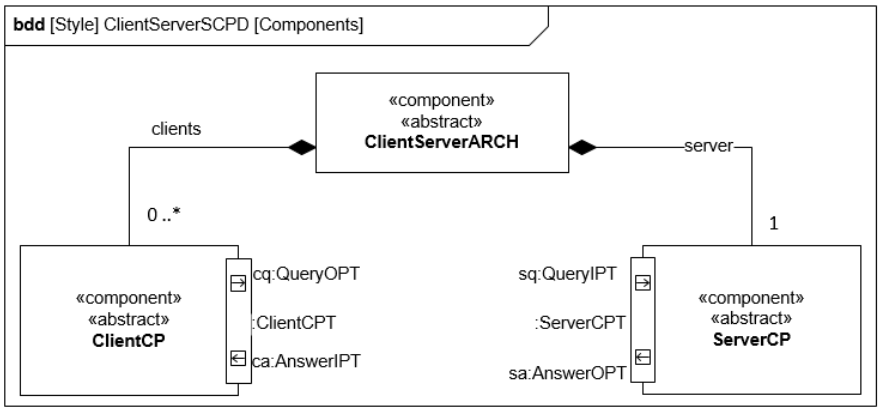


**Figure 5. The *Client-Server* component definitions.**

The *ClientPT* port, depicted in ***Figure 6***, is composed by a *QueryOPT out* port and an *AnswerIPT in* port. Both ports allow values of any type. The *ServerPT* port, shown in ***Figure 7***, is composed by an *AnswerOPT out* port and a *QueryIPT in* port. Both ports allow values of any type.

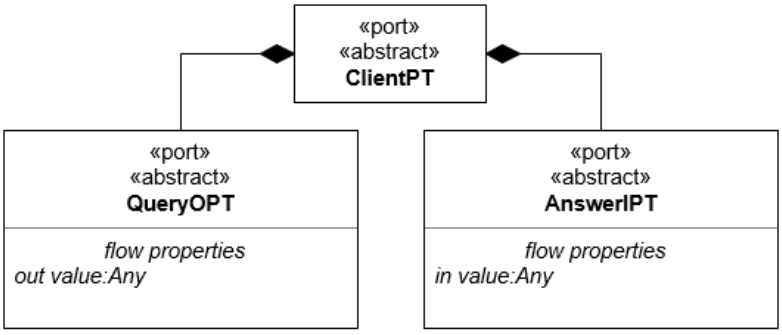


**Figure 6. The *ClientPT* composite port definitions.**

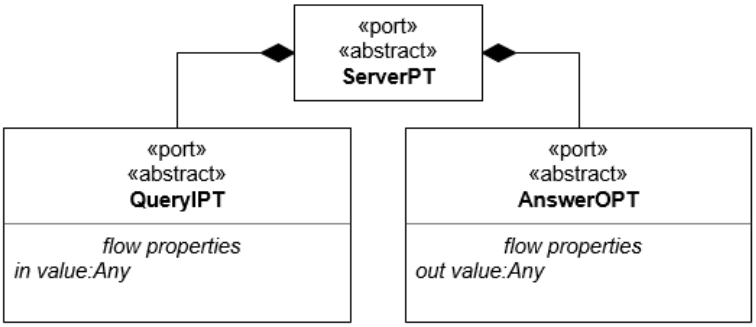


**Figure 7. The *ServerPT* composite port definition.**

In the Client-Server architectural style definition, we must also define the *ClientServerConnectorCN* connector, illustrated in ***Figure 8***. This connector is composed of two connectors: *ClientServerQueryCN* and *ClientServerAnswerCN*, each one is linked to the composite ports of the *ClientCP* and *ServerCP* components.

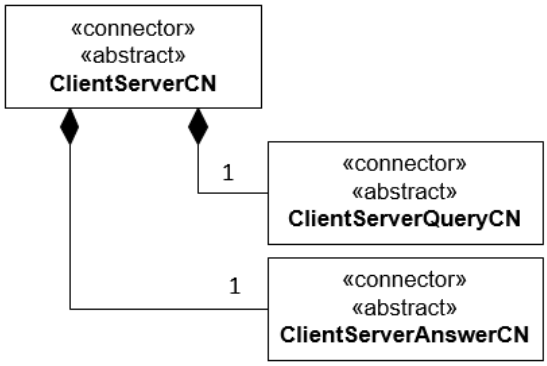


**Figure 8. The *ClientServerCN* composite connector definition.**

The *ClientServerQueryCN* connector, depicted in ***Figure 9***, has two participants ports: *client:QueryOPT* and *server:QueryPortIPT*. The connector conveys data of any type from the client port to the server port.

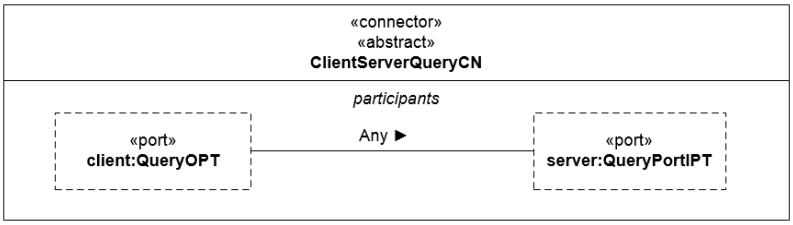


**Figure 9. The *ClientServerQueryCN* connector definition.**

The *ClientServerAnswerCN* connector, illustrated in ***Figure 10***, has two participants ports: *client:AnswerOPT* and *server:QueryAnswerIPT*. The connector conveys data of any type from the server port to the client port.

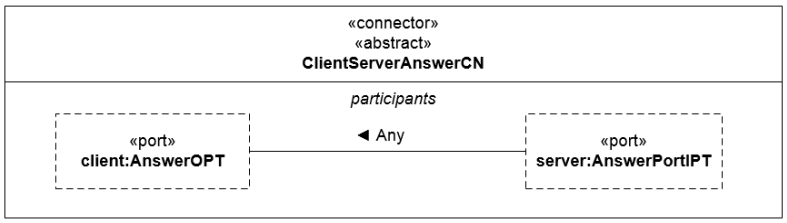


**Figure 10. The *ClientServerAnswerCN* connector definition.**

## 14.4 Client-Server Behavioral Viewpoint

We can specify the behavior of a Client-Server style using activity diagrams specifying the behavior of the connectors and the protocols of the ports.

We define the behavior of the *ClientServerCN* connector by defining the behavior of the two compound connectors.

We define the behavior of the *ClientServerQueryCN* connector using the activity diagram (*ClientServerQueryAC)*, shown in ***Figure 11***. The connector sends to its *out* pin any data received in its *in* pin.

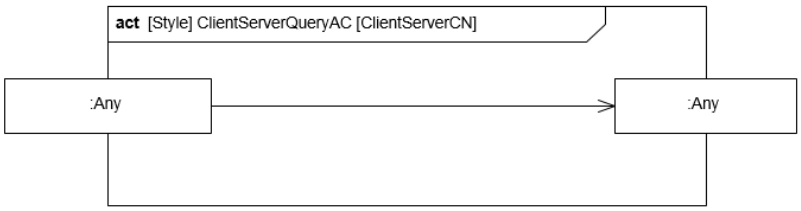


**Figure 11. The *ClientServerQueryCN* connector behavior specification.**

We define the behavior of the *ClientServerAnswerCN* connector using the activity diagram (*ClientServerAnswerAC*), illustrated in ***Figure 12***. The connector sends to its *out* pin any data received in its *in* pin.

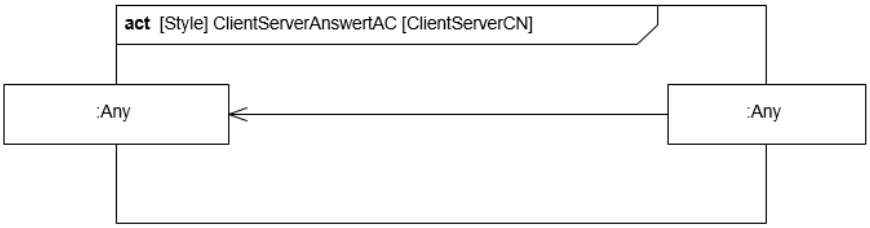


**Figure 12. The *ClientServerAnswerCN* connector behavior specification.**

We define the protocols of the ports using activity diagrams. Since we have two composite ports, we define the protocol of the composite port and the protocols of the individual ports. ***Figure 13*** shows the specification of the protocol of the *ClientPT* port.

The *QueryOPT* internal port initiates by sending a value of any type to its out pin and it waits until the *AnswerIPT* receives data. The *AnswerIPT* internal port receives data from its in pin and the gives the control to the QueryOPT port.

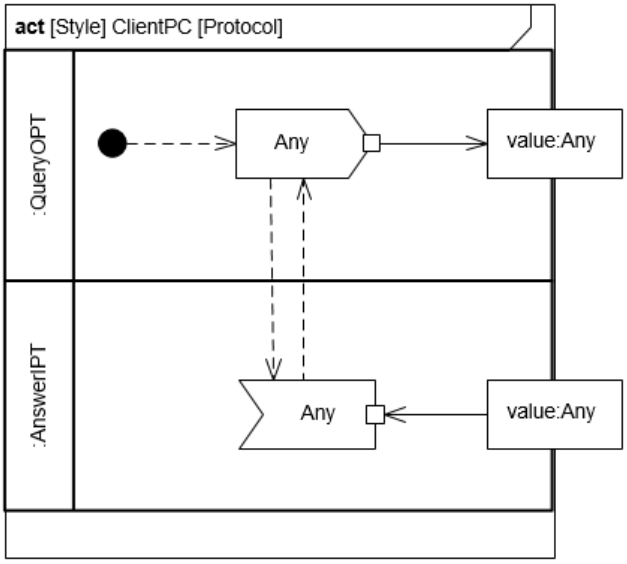


**Figure 13. The specification of the *ClientPC* protocol of the *ClientPT* port.**

***Figure 14*** shows the specification of the protocol of the *ServerPT* port. The *QueryIPT* internal port initiates by receiving a value of any type from its *in* pin.

After that, the control is given to the *AnswerOPT* internal port to send data (the answer) to its *out* pin. Finally, it the gives back the control to the *QueryIPT* port.

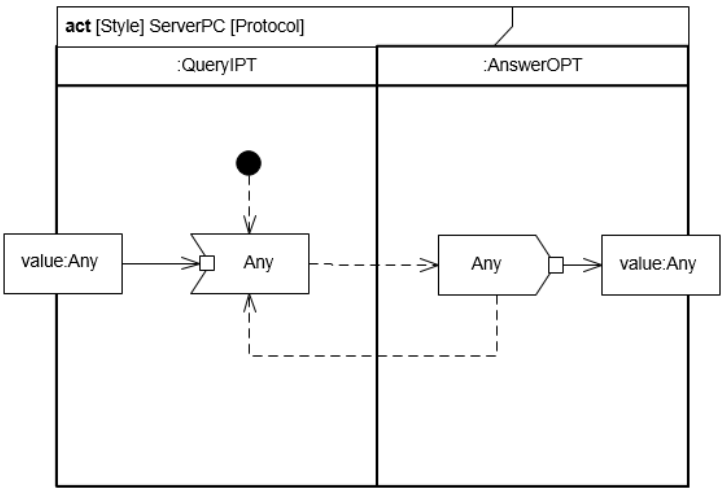


**Figure 14. The specification of the *ServerPC* protocol of the *ServerPT* port.**

We now present the specification the protocol of the individual internal ports of the previous composite ports.

***Figure 15*** shows the internal ports of *ClientPT*. The *QueryOPT* internal port initiates by sending a value of any type to its *out* pin, and then it waits to send another value. The *AnswerIPT* internal port initiates by receiving data from its *in* pin, and then it waits to receive another value.

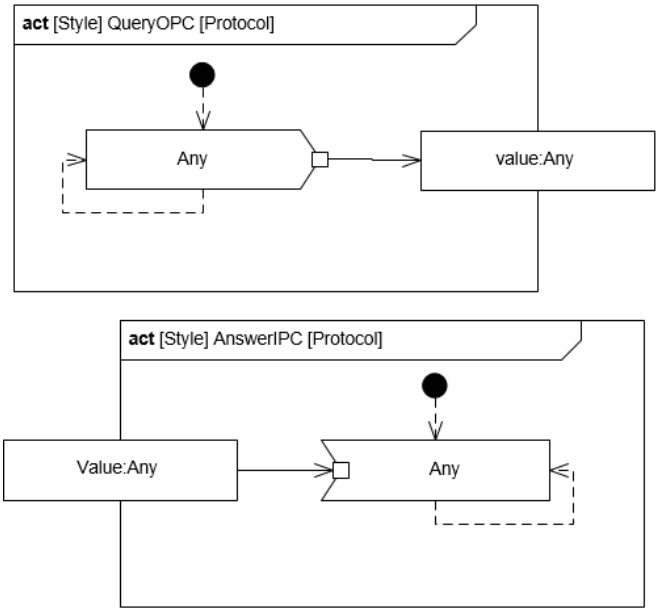


**Figure 15. The specification of the protocols of the *QueryIPT* and *AnswerOPT* ports.**

***Figure 16*** shows the internal ports of *ServerPT*. The *QueryIPT* internal port initiates by receiving a value of any type from its *in* pin, and then it waits to receive another value. The *AnswerOPT* internal port initiates by sending data to its *out* pin, and then it waits for another value to send.

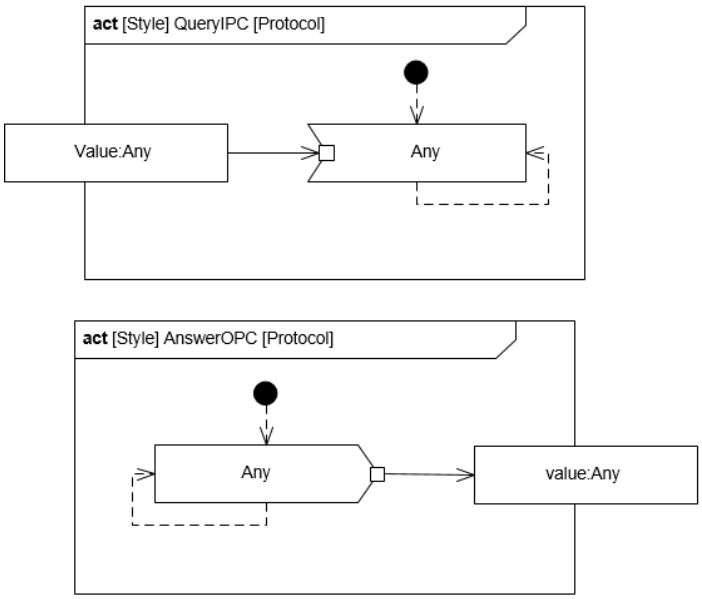


**Figure 16. The specification of the protocols of the *QueryOPT* and *AnswerPT* ports.**

## 14.5 Summary

In this chapter, you learned:

- the Client-Server architectural style;
- the elements, structure and behavior of the Client-Server style.

You learnt how to:

- apply the Client-Server style in an architecture design.

## 14.6 For further reading

Clements, P.; Bachmann, L.; Garlan, D.; Ivers, J.; Little, R.; Merson, P.; Nord, R. Documenting Software Architecture: Views and Beyond. SEI Series in Software Engineering. (2003)

Buschmann, F., Meunier, R., Rohnert, H. Sommerlad, P., Michael Stal, M. Pattern-Oriented Software Architecture Volume 1: A System of Patterns. Vol. 1. Wiley (1996).

Vogel, O.; Arnold, I.; Chughtai, A.; Kehrer, T. Software Architecture: A Comprehensive Framework and Guide for Practitioners. Springer (2011)